



COMPLETE BEGINNER-TO-INTERVIEW HANDBOOK

Complete AI Agents & MCP Handbook

A Beginner's Guide — Questions & Answers

*"Everything you need to understand, build, and interview on AI Agents
and the Model Context Protocol"*

CREATED BY

The Handbook Team

@kartikmungole

CONTENTS

Table of Contents

Your complete map to Agents and the Model Context Protocol

i

Note: This guide is written in a simple **Question & Answer format** — the same questions a curious learner (and an interviewer) would naturally ask. No jargon without explanation.

PART I — AI AGENT FUNDAMENTALS

Ch 1	What is an AI Agent?
Ch 2	AI vs Automation vs Agents
Ch 3	Agent Architecture & Components
Ch 4	Reasoning Patterns
Ch 5	Planning & Task Decomposition
Ch 6	Memory Systems in Agents
Ch 7	Tools & Function Calling

PART II — MODEL CONTEXT PROTOCOL (MCP)

Ch 8	Introduction to MCP
Ch 9	MCP Architecture
Ch 10	MCP Core Primitives
Ch 11	MCP Transports
Ch 12	Building an MCP Server
Ch 13	MCP Security & Permissions

PART III — FRAMEWORKS & ECOSYSTEM

Ch 14	Popular Agent Frameworks
Ch 15	Multi-Agent Systems & Orchestration
Ch 16	RAG and Agents
Ch 17	Agent Evaluation & Observability
Ch 18	Agent Safety & Guardrails

PART IV — PRACTICAL APPLICATION

Ch 19	Real-World Use Cases
Ch 20	Career Roadmap & Practical Tips

INTERVIEW PREPARATION

Ch 21	100 Interview Questions & Answers (10 categories)
-------	---

APPENDIX

A	Glossary of Key Terms
B	Further Resources & Roadmap

How to Use This Book

Legend and structure for the pages ahead



A question a beginner or interviewer would ask



A clear, plain-English answer



Tip — helpful advice or shortcut



Warning — an important caution



Note — extra context worth knowing



Analogy — a real-world comparison

THIS BOOK IS ORGANIZED IN FOUR PARTS

Part I

AI Agent Fundamentals — architecture, reasoning, memory, tools

Part II

Model Context Protocol — architecture, primitives, building servers, security

Part III

Frameworks & Ecosystem — LangChain, CrewAI, multi-agent systems, RAG, evaluation

Part IV

Practical Application — real use cases, career roadmap, and 100 interview questions

PART I

AI Agent Fundamentals

Understanding what agents are, how they reason, remember, and
act

What is an AI Agent?

Understanding agents and where they sit in the AI landscape

Q What exactly is an AI agent?

- A An AI agent is a system built around an LLM that can perceive its environment, reason about what to do, take actions using tools, and observe the results — repeating this loop until it achieves a goal. Unlike a plain chatbot that only replies with text, an agent can browse the web, run code, call APIs, edit files, and make decisions about what to do next.

♥ **Analogy:** Think of a chatbot as a knowledgeable person you can only talk to through a window. An agent is that same person, but now they can walk out the door, use tools, and go get things done on their own.

Q How is an agent different from a regular LLM call?

- A A single LLM call takes an input and produces an output — it is stateless and one-shot. An agent wraps the LLM in a loop: it lets the model decide which tool to call, executes that tool, feeds the result back to the model, and repeats until the task is done or a stopping condition is met.

LLM CALL VS AGENT

Aspect	Plain LLM Call	AI Agent
Interaction	Single request/response	Multi-step loop
Tools	None	Can call functions, APIs, code
State	Stateless	Maintains memory across steps
Autonomy	None — human decides next step	Agent decides next step

What is an AI Agent?

— continued —

Q What are the core components of an agent?

- A** Every agent, regardless of framework, is built from the same four building blocks: a model (the reasoning engine), a set of tools (actions it can take), memory (context it retains), and a control loop / planner (the logic that decides what to do next). Chapter 3 covers each of these in depth.

Q Why are agents suddenly so popular?

- A** Three things converged: LLMs became reliable enough at reasoning and instruction-following to be trusted with multi-step tasks, function/tool calling became a standard capability across major model providers, and protocols like MCP made it easy to connect models to real-world tools and data without custom glue code for every integration.

Note: Agentic AI is considered one of the biggest shifts in how software is built — instead of writing explicit logic for every case, developers describe a goal and let the model figure out the steps.

Q What is 'agentic' behavior?

- A** Agentic behavior refers to a system exhibiting goal-directed autonomy: taking initiative, adapting its plan when something fails, using tools without being told exactly which one to use, and continuing to work toward an objective across multiple steps rather than stopping after one response.

AI vs Automation vs Agents

Placing agents correctly among related concepts

Q How do agents differ from traditional automation (RPA, scripts)?

- A Traditional automation (like RPA or a cron script) follows a fixed, pre-programmed sequence of steps — it breaks the moment something unexpected happens. An agent uses an LLM to reason about the current situation and decide the next action dynamically, so it can handle novel inputs, recover from errors, and adapt its plan on the fly.

WHERE AGENTS FIT

Rule-based automation

Fixed if/then logic, no reasoning

Machine Learning

Learns patterns from data, but doesn't act

LLM chatbot

Reasons in language, but doesn't act on the world

AI Agent

Reasons AND acts, using tools, in a loop

Q Is every LLM app with a tool an 'agent'?

- A Not necessarily. A system that calls exactly one tool in a fixed order is closer to a 'tool-augmented' app. Most practitioners reserve the term 'agent' for systems where the model itself decides whether to call a tool, which tool to call, and when to stop — i.e. the control flow is driven by the model, not hard-coded by the developer.

AI vs Automation vs Agents

— continued —

Q What is the difference between a workflow and an agent?

A A workflow orchestrates LLM calls and tools through code that a developer defines explicitly (fixed graph of steps). An agent lets the LLM dynamically direct its own process and tool usage, retaining control over how it accomplishes the task. Many production systems are actually a hybrid: a workflow with agentic sub-steps.

 **Tip:** When starting out, prefer the simplest solution that works. Add agentic autonomy only when a task genuinely needs open-ended decision-making — it adds cost, latency, and unpredictability.

Q What is 'autonomy level' in agent design?

A Autonomy level describes how much a system is allowed to decide on its own before checking in with a human — ranging from fully human-in-the-loop (approve every action) to fully autonomous (run unsupervised for long horizons). Most production agents today sit in the middle, with checkpoints for high-risk actions.

Agent Architecture & Components

The building blocks every agent shares

Q What is the 'agent loop'?

- A** The agent loop is the repeating cycle of Perceive → Reason → Act → Observe. The agent perceives the current state (user input, tool results), reasons about what to do next, acts by calling a tool or responding, and observes the outcome — then the cycle repeats until the goal is reached or a limit is hit.

♥ **Analogy:** It's like a chef repeatedly tasting a dish, deciding what's missing, adding an ingredient, and tasting again — instead of following a rigid recipe blindly.

Q What is the role of the 'system prompt' in an agent?

- A** The system prompt defines the agent's persona, goals, constraints, available tools, and behavioral rules. It is the closest thing an agent has to a job description, and is usually the single highest-leverage piece of prompt engineering in the whole system.

Q What is an 'orchestrator' in multi-step agents?

- A** The orchestrator is the component (often just the LLM itself, or a lightweight controller around it) that decides the sequence of steps: which sub-task to tackle next, which tool or sub-agent to delegate to, and when the overall task is complete.

Agent Architecture & Components

— continued —

Q What is the difference between a 'single-agent' and 'multi-agent' architecture?

A A single-agent system uses one LLM instance with a toolset to handle a task end-to-end. A multi-agent system splits the task across several specialized agents (e.g. a researcher, a coder, a reviewer) that communicate and hand off work, which can improve quality on complex tasks at the cost of more coordination overhead.

 **Warning:** Multi-agent systems are harder to debug and more expensive to run. Reach for them only when a single agent with good tools genuinely can't handle the task's complexity.

Q What is a 'stopping condition' and why does it matter?

A A stopping condition tells the agent when to stop looping — e.g. the goal is achieved, a maximum number of steps is reached, or the model decides no further action is needed. Without a clear stopping condition, agents can loop indefinitely, burning tokens and money.

Reasoning Patterns

Chain-of-Thought, ReAct, Tree-of-Thought and Reflexion

Q What is Chain-of-Thought (CoT) prompting?

A Chain-of-Thought is a prompting technique where the model is asked to reason step-by-step before giving a final answer, rather than jumping straight to a conclusion. This exposes intermediate reasoning, which tends to improve accuracy on multi-step problems like math or logic.

Q What is the ReAct pattern?

A ReAct (Reason + Act) interleaves reasoning traces with tool calls: the model reasons about what it needs, takes an action (calls a tool), observes the result, and reasons again — repeating until it can answer. It is the foundation of nearly all modern tool-using agents.

Note: Example ReAct trace: Thought: I need today's weather → Action: call weather_api(city) → Observation: '31°C, sunny' → Thought: I can now answer → Final answer.

Q What is Tree-of-Thought (ToT) reasoning?

A Tree-of-Thought extends Chain-of-Thought by exploring multiple reasoning branches in parallel, evaluating which paths look promising, and backtracking from dead ends — similar to how a person might sketch out several possible solutions before picking the best one.

Reasoning Patterns

— continued —

Q What is 'Reflexion' or self-critique in agents?

A Reflexion is a pattern where an agent evaluates its own output or action outcome, generates a critique of what went wrong, and uses that critique to try again — giving the agent a form of iterative self-improvement within a single task, without retraining the underlying model.

Q Why does step-by-step reasoning improve tool use?

A Explicit reasoning forces the model to state its assumptions and intended action before acting, which makes errors easier to catch (either by the model itself or a human reviewing logs), and reduces the chance of the model jumping to an irrelevant tool call.

 **Tip:** When building agents, log the full reasoning trace, not just the final answer — it is the single most useful thing for debugging why an agent made a bad decision.

Planning & Task Decomposition

Breaking big goals into steps an agent can execute

Q What is task decomposition?

- A Task decomposition is the process of breaking a large, ambiguous goal (e.g. 'launch a marketing campaign') into smaller, concrete, executable sub-tasks (e.g. 'draft copy', 'generate images', 'schedule posts') that the agent can tackle one at a time.

Q What is the difference between 'plan-then-execute' and 'reactive' agents?

- A A plan-then-execute agent first drafts a full multi-step plan, then works through it (revising if a step fails). A reactive agent decides only its very next action at each step, without committing to a full plan upfront. Plan-then-execute is more predictable; reactive is more adaptive to surprises.

PLANNING STYLES

Style	Pros	Cons
Plan-then-execute	Predictable, easy to review	Brittle if early assumptions are wrong
Reactive (step-by-step)	Adapts well to surprises	Can lose sight of the overall goal

Q What is a 'sub-goal' and why decompose at all?

- A A sub-goal is an intermediate objective that contributes to the final goal. Decomposing into sub-goals lets an agent make steady, verifiable progress, use the right tool for each narrow piece of work, and recover from a failed sub-task without redoing the entire job.

Planning & Task Decomposition

— continued —

Q How do agents handle failure during planning?

A Well-designed agents treat a failed action as new information rather than a dead end: they observe the error, reason about why it happened, and either retry with adjusted parameters, pick a different tool, or escalate to a human — instead of blindly repeating the same failed action.

Memory Systems in Agents

Short-term, long-term, and episodic memory

Q Why do agents need memory beyond the context window?

- A** An agent's context window is limited and resets between sessions, but many real tasks span hours, days, or many separate conversations. External memory lets an agent recall past interactions, learned facts, and user preferences that no longer fit — or never fit — inside a single context window.

Q What is 'short-term' vs 'long-term' memory in an agent?

- A** Short-term memory is simply the current context window — everything visible to the model right now. Long-term memory is external storage (a database, vector store, or file) that the agent explicitly writes to and retrieves from across sessions, surviving beyond any single conversation.

♥ **Analogy:** Short-term memory is like your working memory while reading a page. Long-term memory is like your notebook — you have to deliberately write things down and look them up later.

Q What is 'episodic' memory in agents?

- A** Episodic memory stores records of specific past events or interactions ('on March 3rd the user asked me to always use metric units'), as opposed to semantic memory, which stores general facts and knowledge independent of when they were learned.

Memory Systems in Agents

— continued —

Q What is a vector store and how is it used for agent memory?

A A vector store holds text converted into numerical embeddings that capture semantic meaning, so an agent can search for 'similar' past memories or documents by meaning rather than exact keyword match — this is the standard mechanism behind most long-term agent memory and Retrieval-Augmented Generation (RAG).

Q What is 'context management' and why is it a core agent skill?

A Context management is deciding what information to keep, summarize, or drop from the context window as a task grows — since stuffing everything in eventually exceeds the window or drowns the model in irrelevant detail. Good agents summarize old steps and only keep the most relevant memory retrieved for the current step.

 **Warning:** A model attending to a huge, cluttered context tends to lose track of instructions given earlier — this is sometimes called 'context rot'. Keep prompts focused.

Tools & Function Calling

How agents actually take action in the world

Q What is 'function calling' / 'tool calling'?

- A** Function calling is a model capability where, instead of only generating text, the model can output a structured request to call a specific function with specific arguments (e.g. `get_weather(city='Bengaluru')`). The developer's code executes that function and returns the result back to the model.

Q What makes a good tool description?

- A** A good tool description clearly states what the tool does, when to use it, what each parameter means, and what the tool returns — written the same way you'd explain it to a new colleague. Vague or overlapping tool descriptions are the single most common cause of an agent picking the wrong tool.

 **Tip:** Give tools names and descriptions from the model's point of view, e.g. `'search_orders: Look up a customer's past orders by email address'` rather than a generic name like `'db_query'`.

Q What is a 'tool schema'?

- A** A tool schema is a structured definition (typically JSON Schema) listing a tool's name, description, and parameters with their types and constraints. The model is given the list of available schemas and chooses which tool(s), if any, to call based on them.

Tools & Function Calling

— continued —

Q What happens if a tool call fails?

- A** The error message is returned to the model as the 'observation' just like a successful result would be. A well-built agent reasons about the error and adjusts — retrying with corrected arguments, trying a different tool, or telling the user it could not complete the action — rather than crashing.

Q What is the difference between a 'tool' and an 'MCP tool'?

- A** A tool in general is just a function the model can call, usually wired up manually in a specific application's code. An MCP tool is a tool exposed through the Model Context Protocol's standard interface, so any MCP-compatible client can discover and call it without custom integration code — this is the focus of Part 2 of this book.

PART II

Model Context Protocol (MCP)

The open standard connecting AI models to tools and data

Introduction to MCP

What the Model Context Protocol is and why it exists

Q What is MCP (Model Context Protocol)?

- A MCP is an open, standardized protocol (introduced by Anthropic in late 2024) that defines how AI applications connect to external data sources and tools. Instead of every app writing custom, one-off integrations for every tool, MCP defines a common interface so any MCP client can talk to any MCP server.

♥ **Analogy:** MCP is often compared to USB-C for AI: instead of a different cable (integration) for every device, one standard connector lets any compatible client plug into any compatible server.

Q What problem does MCP solve?

- A Before MCP, connecting an LLM app to N different tools/data sources typically required N different custom integrations, and every new AI app had to reimplement all of them again. MCP turns this into an $M \times N$ problem into an $M+N$ problem: any MCP client works with any MCP server, so integrations are written once and reused everywhere.

Q Who created MCP and is it open?

- A MCP was open-sourced by Anthropic and is now a community-driven, vendor-neutral specification with a public GitHub repository, SDKs in multiple languages, and adoption across many AI companies and developer tools beyond Anthropic's own products.

i **Note:** Being open-source means anyone can build an MCP server or client without needing permission or a partnership agreement — this is a major reason for its rapid adoption.

Introduction to MCP

— continued —

Q What is the difference between MCP and a plain API?

A A plain REST/GraphQL API is designed for developers to integrate against with custom code for each endpoint. MCP is designed specifically for LLMs: it standardizes how capabilities are described (so a model can understand what a tool does), discovered dynamically, and invoked in a way any MCP-aware client already knows how to handle.

Q What is the difference between MCP and function calling?

A Function calling is the model-level capability to request a structured tool call. MCP is the protocol that standardizes how those tools (and other capabilities) are exposed, discovered, and connected between an application and external servers — MCP servers ultimately surface tools that get used via function calling under the hood.

MCP Architecture

Hosts, Clients, and Servers explained

Q What are the three main roles in MCP's architecture?

- A MCP defines Hosts (the AI application the user interacts with, e.g. Claude Desktop or an IDE), Clients (a component inside the host that maintains a 1:1 connection to a specific server), and Servers (lightweight programs that expose tools, resources, and prompts to clients).

MCP ROLES

Role	Description	Example
Host	The user-facing AI application	Claude Desktop, an IDE, a custom agent app
Client	Manages one connection to one server	Built into the host
Server	Exposes tools/resources/prompts	A GitHub MCP server, a database MCP server

Q Why does MCP use a 1:1 client-server connection model?

- A Each client instance connects to exactly one server, keeping connections isolated and simple to reason about — the host can run many clients in parallel to talk to many servers at once, but each individual connection stays a clean, stateful, one-to-one relationship.

Q How does a host discover what a server can do?

- A When a client connects to a server, it performs an initialization handshake where the server advertises its capabilities (which tools, resources, and prompts it offers). The host then presents these capabilities to the model or the user as available options.

MCP Architecture

— continued —

Q Can one host connect to multiple MCP servers at once?

A Yes — this is one of MCP's core strengths. A single host application can run multiple clients simultaneously, each connected to a different server (e.g. one for GitHub, one for a database, one for Slack), giving the model access to a combined toolbox from many sources at once.

MCP Core Primitives

Tools, Resources, Prompts, and Sampling

Q What are the four core primitives an MCP server can expose?

- A An MCP server can expose Tools (actions the model can invoke, like function calls), Resources (read-only data the host can attach as context, like files or records), Prompts (reusable, parameterized prompt templates), and Sampling (a way for the server to request the client's LLM to generate a completion).

MCP PRIMITIVES

Tools

Model-controlled actions, e.g. `create_issue()`

Resources

App-controlled context, e.g. a file's contents

Prompts

User-controlled templates, e.g. `/summarize-pr`

Sampling

Server asks the client's LLM to generate text

Q What is the difference between a 'tool' and a 'resource' in MCP?

- A Tools are actions the model decides to invoke dynamically during reasoning (model-controlled). Resources are pieces of data — like a file, a database row, or a log — that the host application attaches to the context, typically at the user's or app's discretion rather than the model's (application-controlled).

Q What is an MCP 'prompt' primitive used for?

- A MCP prompts are pre-written, reusable prompt templates that a server exposes for a host's UI to surface — for example, a slash command like `/review-pr` that a user explicitly triggers, which the server fills in with the right context and instructions.

MCP Core Primitives

— continued —

Q What is 'sampling' in MCP and why is it unusual?

A Sampling lets an MCP server request that the connected client's LLM generate a completion — effectively reversing the normal direction of control. This lets a server perform its own agentic sub-tasks (e.g. summarizing data before returning it) without needing its own separate API key or model access.

 **Warning:** Because sampling lets a server ask the model to generate content, clients typically require explicit human approval before honoring a sampling request, to prevent abuse.

MCP Transports

stdio, HTTP, and how messages actually move

Q What transport mechanisms does MCP support?

- A MCP primarily supports two transports: stdio (standard input/output), used when the client launches the server as a local subprocess, and Streamable HTTP, used when the server runs remotely and the client connects over the network — both carry the same underlying JSON-RPC 2.0 messages.

Q When should you use stdio vs HTTP transport?

- A Use stdio for local integrations where the server runs on the same machine as the host (e.g. a local file-system or git server) — it's simple and has no networking or auth concerns. Use Streamable HTTP when the server needs to run remotely, be shared across multiple users, or scale independently of the client.

TRANSPORT COMPARISON

Transport	Best for	Auth needed?
stdio	Local tools on the same machine	No
Streamable HTTP	Remote / shared / cloud servers	Usually yes (OAuth, API keys)

Q What underlying message format does MCP use?

- A MCP messages are encoded as JSON-RPC 2.0 — a lightweight remote procedure call protocol using JSON. This gives MCP a well-understood, language-agnostic wire format for requests, responses, and notifications between client and server.

MCP Transports

— continued —

- Q** What is a 'notification' in MCP, as opposed to a request?
- A** A request expects a response (e.g. 'call this tool and give me the result'). A notification is a one-way message with no response expected — used for things like a server telling the client that its list of available tools has changed.

Building an MCP Server

A practical walkthrough of server development

Q What are the basic steps to build an MCP server?

- A** At a high level: (1) pick an SDK for your language (Python, TypeScript, and others are officially supported), (2) define your tools/resources/prompts with clear names, descriptions, and schemas, (3) implement the handler logic for each, and (4) choose a transport (stdio for local, HTTP for remote) and run the server.

Q What does a minimal tool definition look like conceptually?

- A** A tool definition needs a unique name, a human-and-model-readable description, an input schema (parameter names, types, and which are required), and a handler function that executes the actual logic and returns a result — for example, a 'get_forecast' tool taking a city name and returning weather text.

Q What common mistakes do developers make building their first MCP server?

- A** Common pitfalls include exposing too many overlapping tools (confusing the model about which to pick), writing vague descriptions, not validating tool inputs, returning huge unstructured blobs of data instead of concise results, and forgetting to handle and clearly report errors back to the model.

 **Tip:** Start with a small number of well-named, well-described tools. It is far easier to add tools later than to fix a model that keeps misusing a confusing toolset.

Building an MCP Server

— continued —

Q How should errors be returned from an MCP tool?

- A** Tool errors should be returned as a structured error result within the protocol (not by crashing the server), with a clear, human-readable message explaining what went wrong — this lets the connected model reason about the failure and decide how to recover, just like any other tool observation.

Q Can an MCP server call other MCP servers or external APIs?

- A** Yes. An MCP server is just a regular program — internally, it is free to call any external API, database, or even other MCP servers. From the client's perspective, it only sees the clean tool interface the server chooses to expose.

MCP Security & Permissions

Trust boundaries, consent, and safe deployments

Q Why is security especially important for MCP servers?

- A** MCP servers can grant a model the ability to read sensitive data or take real-world actions (send emails, modify files, move money). Because the model decides when to call a tool, a server must assume it may receive unexpected or even adversarial inputs and enforce its own safety limits rather than trusting the model completely.

 **Warning:** Never expose a tool that can perform an irreversible, high-impact action (e.g. deleting a production database) without a human confirmation step in between.

Q What is 'human-in-the-loop' consent in MCP hosts?

- A** Most MCP hosts require the user to explicitly approve a tool call before it executes, especially for actions with side effects — showing what tool is being called with what arguments, so the user can catch mistakes or unwanted behavior before anything happens.

Q How should authentication work for remote MCP servers?

- A** Remote MCP servers over HTTP typically use OAuth 2.1 or similar token-based authentication, so the server can verify who is connecting and scope what data or actions that specific user is allowed to access — rather than sharing one global credential across all users.

MCP Security & Permissions

— continued —

Q What is 'prompt injection' and why does it matter for MCP?

- A** Prompt injection is when malicious instructions are hidden inside content the model reads (a webpage, a file, a tool's output) in an attempt to hijack the model's behavior. Because MCP tools often return external data straight into the model's context, servers and hosts must treat all returned content as untrusted.

Q What is the principle of least privilege in agent tooling?

- A** An agent or MCP server should only be granted the minimum permissions needed to do its job — e.g. read-only database access if it never needs to write, or a scoped API token instead of an admin key — to limit the damage if the model behaves unexpectedly or is manipulated.

PART III

Frameworks & Ecosystem

The frameworks, patterns, and practices used to build real systems

Popular Agent Frameworks

A map of the tools developers use to build agents

Q What is LangChain and what is it used for?

A LangChain is one of the earliest and most widely used open-source frameworks for building LLM applications, providing abstractions for chains, agents, memory, and tool integration across many model providers and data sources.

Q What is LangGraph?

A LangGraph, built by the LangChain team, models agent workflows as an explicit graph of nodes and edges, giving developers fine-grained control over branching, looping, and state — popular for production agents that need predictable, debuggable control flow.

Q What is LlamaIndex best known for?

A LlamaIndex focuses heavily on data ingestion, indexing, and retrieval — making it a common choice for building Retrieval-Augmented Generation (RAG) pipelines and agents that need to reason over large private document collections.

Q What is CrewAI?

A CrewAI is a framework specifically designed for orchestrating multi-agent 'crews' — teams of agents with defined roles (e.g. researcher, writer, editor) that collaborate on a shared task, with built-in patterns for delegation and hand-offs.

Popular Agent Frameworks

— continued —

Q What is AutoGen?

A AutoGen, from Microsoft Research, is a framework for building multi-agent conversations, where agents (including human-in-the-loop proxies) message each other to collaboratively solve tasks — often used in research and complex automation scenarios.

Q What is the Claude Agent SDK / Claude Code, and how do they relate to agents?

A Claude Code is Anthropic's agentic coding tool, letting developers delegate coding tasks to Claude from the terminal or IDE. The underlying Claude Agent SDK exposes the same agent loop, tool-use, and context-management primitives so developers can build their own custom agents on top of Claude.

FRAMEWORK QUICK REFERENCE

Framework	Best Known For
LangChain / LangGraph	General-purpose agents, graph-based control flow
LlamaIndex	RAG and data-heavy retrieval
CrewAI	Role-based multi-agent teams
AutoGen	Multi-agent conversation patterns
Claude Agent SDK	Production agents on Claude models

Multi-Agent Systems & Orchestration

When and how to coordinate several agents

Q When should you use multiple agents instead of one?

A Multiple agents help when a task naturally splits into distinct specialized roles (e.g. research vs. writing vs. fact-checking), when parallelizing independent sub-tasks would save time, or when a single agent's context would become too cluttered doing everything itself.

 **Warning:** Multi-agent systems multiply cost, latency, and failure modes. Many tasks that seem to need multiple agents can be solved just as well by one agent with better tools and a clearer prompt.

Q What is the 'orchestrator-worker' pattern?

A In this pattern, a lead 'orchestrator' agent breaks the overall goal into sub-tasks and delegates each to specialized 'worker' agents, then collects and synthesizes their results into a final answer — a common, easy-to-reason-about multi-agent structure.

Q How do agents communicate with each other in a multi-agent system?

A Depending on the framework, agents may communicate via structured messages passed through an orchestrator, a shared memory/blackboard both can read and write to, or direct message-passing between agents — each with different trade-offs in coordination complexity and traceability.

Multi-Agent Systems & Orchestration

— continued —

- Q What is a common failure mode in multi-agent systems?**
- A** A frequent failure is 'context loss at hand-off' — when one agent passes work to another without including enough of the original context, causing the second agent to repeat work, misunderstand the goal, or contradict earlier decisions.

RAG and Agents

Retrieval-Augmented Generation as an agent capability

Q What is Retrieval-Augmented Generation (RAG)?

- A** RAG is a technique where relevant documents or data are retrieved from an external knowledge base (often a vector store) and inserted into the model's context before it generates a response — grounding the answer in specific, up-to-date, or private information the model wasn't trained on.

♥ **Analogy:** RAG is like giving an open-book exam: instead of relying purely on memory, the model is handed the exact reference pages relevant to the question before answering.

Q How does 'agentic RAG' differ from traditional RAG?

- A** Traditional RAG performs one retrieval step before generating an answer. Agentic RAG lets the model decide, iteratively, whether it needs to search again, refine its query, pull from a different source, or that it already has enough information — turning retrieval into a tool the agent actively controls.

Q What is 'chunking' and why does it matter for RAG?

- A** Chunking is splitting large documents into smaller pieces before embedding and storing them, since retrieval works best on focused, appropriately-sized passages rather than entire documents. Chunk size and overlap significantly affect retrieval quality.

RAG and Agents

— continued —

Q What is re-ranking in a retrieval pipeline?

A Re-ranking is a second pass that re-scores an initial set of retrieved candidates using a more precise (often more expensive) model, to push the truly most relevant results to the top before they're handed to the LLM — improving answer quality without searching the entire corpus with the expensive model.

Agent Evaluation & Observability

Measuring whether an agent is actually working

Q Why is evaluating agents harder than evaluating a single LLM response?

A An agent's output depends on a whole sequence of decisions — which tools it picked, in what order, with what arguments — so a wrong final answer could stem from many different points of failure. Evaluation needs to look at the full trajectory, not just the final message.

Q What is 'tracing' in the context of agents?

A Tracing is capturing a detailed, step-by-step log of everything an agent did during a run — every reasoning step, tool call, tool result, and model response — so developers can replay and inspect exactly what happened when debugging or auditing behavior.

Q What metrics are commonly used to evaluate agents?

A Common metrics include task success rate (did it achieve the goal), tool-call accuracy (did it pick the right tool with correct arguments), number of steps/cost to completion, and human or model-graded quality scores on the final output.

COMMON AGENT EVAL METRICS

Metric	What it measures
Task success rate	Did the agent complete the goal correctly?
Tool accuracy	Right tool, right arguments, right timing?
Efficiency	Steps, tokens, and time to completion
Groundedness	Are claims backed by retrieved/tool data?

Agent Evaluation & Observability

— continued —

Q What is an 'LLM-as-judge' evaluation?

A LLM-as-judge uses a (often stronger) language model to grade another model's or agent's output against a rubric, since manually grading every run at scale isn't feasible. It's widely used but must be validated against human judgment to make sure the judge model's grading is trustworthy.

Agent Safety & Guardrails

Keeping autonomous systems within safe bounds

Q What is a 'guardrail' in agent systems?

- A** A guardrail is a check — implemented in code, a separate model, or explicit rules — that constrains what an agent is allowed to do, such as blocking certain tool calls, validating outputs before they're shown to a user, or capping how many steps or how much money an agent can spend on a task.

Q Why do agents need spending/step limits?

- A** Because agents operate in a loop and decide their own next steps, a bug or a confusing situation can cause them to loop far longer than intended, burning API costs or taking unintended real-world actions. Hard limits on steps, time, and cost act as a safety net.

 **Warning:** Always set an explicit maximum number of iterations or a timeout for any autonomous agent loop — never let one run “until it decides it's done” without a hard ceiling.

Q What is 'sandboxing' for agent tool execution?

- A** Sandboxing runs an agent's actions (e.g. executing code, browsing the web) inside an isolated environment with restricted permissions, so that even if the agent is manipulated or makes a mistake, the potential damage is contained and cannot affect the host system or other users.

Agent Safety & Guardrails

— continued —

Q What is the difference between input and output guardrails?

A Input guardrails screen what goes into the model (e.g. filtering malicious instructions hidden in retrieved content). Output guardrails screen what the model produces before it's acted on or shown to the user (e.g. checking a generated SQL query isn't a destructive DELETE before executing it).

PART IV

Practical Application

Use cases, career guidance, and 100 interview questions

Real-World Use Cases

Where agents and MCP are delivering value today

Q What are the most common production use cases for AI agents today?

- A Common use cases include coding assistants that read, write, and test code (like Claude Code and Cursor), customer support agents that look up orders and resolve tickets, research agents that gather and synthesize information across many sources, and internal ops agents that automate multi-step business workflows.

AGENT USE CASE SNAPSHOT

Software engineering

Writing, reviewing, debugging, and testing code

Customer support

Looking up accounts, resolving tickets end-to-end

Research & analysis

Gathering, cross-checking, and synthesizing information

Operations & workflow

Multi-step business processes across many tools

Q Why has coding become the flagship use case for agents?

- A Coding tasks have clear, verifiable success signals (tests pass, code compiles), well-defined tools (read/write files, run commands), and a tight feedback loop — making it one of the easiest domains to build reliable, high-value agents in.

Q How are MCP servers being used in real products?

- A Companies expose MCP servers for their products (e.g. project management tools, databases, design tools, CRMs) so that any MCP-compatible AI assistant can read and act on that data on a user's behalf, without the company having to build a custom integration for every AI vendor.

Real-World Use Cases

— continued —

Q What industries are adopting agentic AI fastest?

A Software development, customer support, finance/back-office operations, and research-heavy fields (legal, market research) have moved fastest, largely because these domains have well-defined tasks, existing APIs to hook into, and clear ROI from automating multi-step, tool-heavy work.

Career Roadmap & Practical Tips

What to learn next and how to build your first agent

Q What should a beginner learn first before building agents?

- A** Start with solid prompting fundamentals and a single well-tested tool call, then layer in the agent loop (reasoning + acting), then memory, then multiple tools — resist the urge to jump straight into multi-agent or MCP-heavy systems before the basics are solid.

 **Tip:** Build the smallest possible working agent first — one tool, one clear task — and only add complexity once you understand exactly why the simple version isn't enough.

Q What skills are most valuable for an 'AI agent engineer' role?

- A** Strong prompt engineering, an understanding of LLM reasoning patterns, practical experience with tool/function calling and MCP, ability to design evaluation pipelines, and general software engineering skills (APIs, databases, testing, observability) are all highly valued.

Q What are common pitfalls when shipping agents to production?

- A** Common pitfalls include no spending/step limits, poor tool descriptions causing wrong tool selection, insufficient logging/tracing to debug failures, missing human-in-the-loop checks on risky actions, and evaluating only on a handful of hand-picked examples instead of a representative test set.

Career Roadmap & Practical Tips

— continued —

Q What should I learn next after this book?

A Good next steps include hands-on practice building an MCP server for a real tool you use, studying open-source agent codebases, learning evaluation and observability tooling, and — most importantly — shipping a small real agent and iterating based on where it actually fails.

Note: Chapter 6 (Ch.100 Interview Questions section) of this book is designed as a companion for interview preparation once you've built hands-on experience.

CHAPTER 21

100 Interview Questions & Answers

10 categories × 10 questions — everything you need to walk into an
AI Agents / MCP interview prepared

INTERVIEW PREP

AI Agents Fundamentals

Questions 1–6 of 100

Q1 What is an AI agent, in one sentence?

A A system that uses an LLM to perceive, reason, decide, and act using tools in a loop to accomplish a goal, rather than just producing a single text response.

Q2 How does an agent differ from a chatbot?

A A chatbot only converses in text; an agent can take real actions via tools (search, code execution, API calls) and decides its own next steps across multiple turns.

Q3 What are the four core components of an agent?

A The model (reasoning engine), tools (actions available), memory (retained context), and a control loop/planner that decides the next step.

Q4 What is the 'agent loop'?

A The repeating cycle of perceive → reason → act → observe, continuing until the goal is met or a stopping condition is reached.

Q5 Give an example of a stopping condition for an agent.

A A maximum number of steps, a time/cost budget, or the model itself determining the task is complete and returning a final answer.

Q6 What is the difference between an agent and a workflow?

A A workflow follows a developer-defined fixed sequence of steps; an agent lets the model dynamically decide the sequence and which tools to use.

AI Agents Fundamentals

Questions 7–10 of 100

Q7 Why is autonomy risky in agent systems?

- A** Because the agent decides its own actions, mistakes or manipulation can lead to unintended, costly, or harmful actions if not bounded by guardrails and limits.

Q8 What is meant by 'human-in-the-loop'?

- A** A design where a human must review or approve certain agent actions (especially risky or irreversible ones) before they are executed.

Q9 What is agentic RAG?

- A** A retrieval pattern where the agent itself decides when and what to search for, potentially issuing multiple refined queries, rather than retrieving once up front.

Q10 Why did agentic AI become practical only recently?

- A** Because LLMs reached sufficient reliability at multi-step reasoning and tool use, and standardized tool-calling and protocols like MCP removed the need for custom integrations.

INTERVIEW PREP

Agent Architecture & Reasoning

Questions 11–16 of 100

- Q11 What is Chain-of-Thought prompting?**
- A Asking the model to reason step-by-step before giving a final answer, which improves accuracy on multi-step problems by exposing intermediate reasoning.
- Q12 What is the ReAct pattern?**
- A A pattern that interleaves reasoning ('Thought') with tool calls ('Action') and their results ('Observation'), repeating until the model can answer.
- Q13 What is Tree-of-Thought reasoning?**
- A An extension of chain-of-thought that explores multiple reasoning branches in parallel and can backtrack from unpromising paths.
- Q14 What is Reflexion / self-critique?**
- A A pattern where the agent evaluates its own output, generates a critique, and retries using that feedback, without any model retraining.
- Q15 What is 'plan-then-execute' vs 'reactive' agent design?**
- A Plan-then-execute drafts a full multi-step plan upfront then executes it; reactive agents decide only the next single action at each step.
- Q16 Why should reasoning traces be logged?**
- A They make it possible to debug why an agent took a particular action, since the final answer alone doesn't reveal the decision path.

Agent Architecture & Reasoning

Questions 17–20 of 100

Q17 What is an 'orchestrator' agent?

A A lead agent that decomposes a task and delegates sub-tasks to specialized worker agents or tools, then synthesizes their results.

Q18 What causes an agent to loop unnecessarily?

A Poor stopping conditions, ambiguous tool descriptions causing repeated wrong calls, or the model failing to recognize the task is already complete.

Q19 What is task decomposition?

A Breaking a large or ambiguous goal into smaller, concrete, executable sub-tasks the agent can tackle sequentially or in parallel.

Q20 How should an agent handle a failed tool call?

A Treat the error as new information: reason about the cause, then retry with corrected input, try a different approach, or escalate to a human.

INTERVIEW PREP

Memory & Context

Questions 21–27 of 100

Q21 What is the difference between short-term and long-term agent memory?

- A Short-term memory is the current context window; long-term memory is external storage (database, vector store, file) retrieved across sessions.

Q22 What is episodic memory in an agent?

- A Memory of specific past events or interactions, as opposed to general facts (semantic memory) independent of when they were learned.

Q23 What is a vector store used for?

- A Storing text as embeddings so an agent can retrieve semantically similar content by meaning rather than exact keyword match.

Q24 What is 'context rot'?

- A The tendency for a model to lose track of earlier instructions as the context window becomes long and cluttered with less relevant information.

Q25 Why summarize old steps in a long agent run?

- A To keep the context focused and within limits, preserving the key facts while discarding low-value detail from earlier steps.

Q26 What is chunking in the context of memory/RAG?

- A Splitting large documents into smaller passages before embedding and storage, since retrieval works best on focused, right-sized chunks.

Q27 What is the risk of an unbounded context window?

- A Cost and latency increase, and the model can be distracted by irrelevant history, reducing answer quality even if the window technically fits.

Memory & Context

Questions 28–30 of 100

Q28 **How does an agent decide what to write to long-term memory?**

A Typically via an explicit tool call or heuristic — e.g. storing user preferences, key facts, or task outcomes flagged as worth retaining.

Q29 **What is the difference between memory and context?**

A Context is what's currently visible to the model in a single call; memory is a persistent store the agent reads from and writes to across calls/sessions.

Q30 **Why can too much retrieved context hurt an answer?**

A Irrelevant or redundant retrieved passages can dilute the model's attention and even introduce contradictory information, lowering answer quality.

INTERVIEW PREP

Tools & Function Calling

Questions 31–36 of 100

Q31 What is function calling?

- A A model capability to output a structured request to call a specific function with arguments; the developer's code executes it and returns the result.

Q32 What makes a tool description effective?

- A Clear language about what the tool does, when to use it, and what each parameter means — written as if explaining it to a new colleague.

Q33 What is a tool schema?

- A A structured (typically JSON Schema) definition of a tool's name, description, and parameters with types and constraints.

Q34 What happens when a tool call returns an error?

- A The error is passed back to the model as an observation, letting the model reason about it and decide how to recover.

Q35 Why can overlapping tool descriptions cause problems?

- A The model may struggle to choose the correct tool when two tools sound similar, leading to wrong or inefficient tool selection.

Q36 What is the difference between a tool and an MCP tool?

- A A tool is any function a model can call; an MCP tool is one exposed via the standardized Model Context Protocol interface, reusable across clients.

Tools & Function Calling

Questions 37–40 of 100

Q37 How should large tool outputs be handled?

- A Return concise, structured results rather than huge raw blobs, since oversized outputs waste context and can degrade reasoning quality.

Q38 What is 'parallel tool calling'?

- A A model capability to request multiple independent tool calls in a single turn, which the application can execute concurrently to save time.

Q39 Why validate tool inputs server-side?

- A Because the model can generate malformed or unexpected arguments; server-side validation prevents crashes and unsafe operations.

Q40 What is a common cause of an agent picking the wrong tool?

- A Vague, overlapping, or missing descriptions that don't clearly differentiate when each tool should be used.

INTERVIEW PREP

MCP Fundamentals

Questions 41–47 of 100

Q41 What does MCP stand for?

- A Model Context Protocol — an open standard for connecting AI applications to external tools and data sources.

Q42 What problem does MCP solve?

- A It replaces the need for N custom integrations per app with one standard interface, turning an M×N integration problem into M+N.

Q43 Who introduced MCP?

- A Anthropic introduced and open-sourced MCP, and it has since become a community-driven, vendor-neutral specification.

Q44 What is the USB-C analogy for MCP?

- A Just as USB-C lets any compatible device plug into any compatible port, MCP lets any compatible client connect to any compatible server.

Q45 Is MCP tied to a specific model provider?

- A No — MCP is open and model-agnostic; any AI application or model provider can implement MCP clients or servers.

Q46 How does MCP differ from a typical REST API?

- A MCP is purpose-built for LLMs: it standardizes capability discovery and invocation for models, rather than requiring bespoke per-endpoint integration code.

Q47 What underlying protocol do MCP messages use?

- A JSON-RPC 2.0, a lightweight, language-agnostic remote procedure call format.

MCP Fundamentals

Questions 48–50 of 100

Q48

What is the relationship between MCP and function calling?

- A MCP standardizes how tools are exposed and discovered; the model still ultimately performs function calling to invoke the tools MCP surfaces.

Q49

Name one practical benefit of MCP for developers.

- A They can write one MCP server for their product and have it work with any MCP-compatible AI client, instead of maintaining separate integrations.

Q50

What kind of systems commonly implement MCP servers today?

- A Developer tools, project management platforms, databases, design tools, CRMs, and file systems, among many others.

INTERVIEW PREP

MCP Architecture & Implementation

Questions 51–57 of 100

- Q51 What are the three roles in MCP's architecture?**
- A Host (the AI application), Client (manages one connection to one server), and Server (exposes tools, resources, and prompts).

- Q52 What is the client-server connection model in MCP?**
- A Each client maintains a 1:1 stateful connection to one server; a host can run multiple clients to connect to multiple servers simultaneously.

- Q53 What are the four core MCP primitives?**
- A Tools, Resources, Prompts, and Sampling.

- Q54 What is the difference between a Tool and a Resource in MCP?**
- A Tools are model-controlled actions invoked dynamically; Resources are application-controlled, read-only data attached as context.

- Q55 What is an MCP Prompt primitive?**
- A A reusable, parameterized prompt template a server exposes, often triggered explicitly by the user (e.g. a slash command).

- Q56 What is Sampling in MCP?**
- A A mechanism letting a server request the connected client's LLM to generate a completion, reversing the usual direction of control.

- Q57 What transports does MCP support?**
- A stdio for local subprocess servers, and Streamable HTTP for remote servers over the network.

MCP Architecture & Implementation

Questions 58–60 of 100

- Q58 When would you choose stdio over HTTP transport?**
- A** When the server runs locally on the same machine as the host and doesn't need to be shared across users or scaled independently.
- Q59 What is required for authenticating remote MCP servers?**
- A** Typically OAuth 2.1 or token-based authentication, so the server can verify identity and scope access appropriately.
- Q60 What are common mistakes when building a first MCP server?**
- A** Too many overlapping tools, vague descriptions, unvalidated inputs, oversized outputs, and poor error reporting back to the model.

INTERVIEW PREP

Agent Frameworks

Questions 61–67 of 100

Q61 What is LangChain used for?

- A A widely used framework providing abstractions for chains, agents, memory, and tool/data integration across LLM providers.

Q62 What is LangGraph best suited for?

- A Modeling agent workflows as an explicit graph for fine-grained, predictable control over branching and state — good for production systems.

Q63 What does LlamaIndex specialize in?

- A Data ingestion, indexing, and retrieval, making it a common choice for building RAG pipelines.

Q64 What is CrewAI designed for?

- A Orchestrating multi-agent 'crews' with defined roles that collaborate on shared tasks.

Q65 What is AutoGen used for?

- A Building multi-agent conversational systems, including human-in-the-loop proxies, for collaborative task solving.

Q66 What is the Claude Agent SDK?

- A A developer SDK exposing Claude's agent loop, tool-use, and context-management primitives for building custom production agents.

Q67 Why might a team choose a graph-based framework like LangGraph over a purely conversational one?

- A Graph-based frameworks give explicit, debuggable control over branching and state, which is valuable for predictable production behavior.

Agent Frameworks

Questions 68–70 of 100

Q68

What's a risk of picking a heavy framework for a simple agent task?

- A Unnecessary complexity and overhead — a simple agent might be built and maintained more easily with direct API calls and a thin custom loop.

Q69

What should you evaluate when choosing an agent framework?

- A Ecosystem/tool integrations, debugging and observability support, flexibility of control flow, and how well it matches your task's complexity.

Q70

Are agent frameworks required to build an agent?

- A No — an agent can be built directly against a model's API with a simple loop and tool definitions; frameworks add convenience, not a hard requirement.

INTERVIEW PREP

Multi-Agent Systems

Questions 71–77 of 100

- Q71 When is a multi-agent system justified over a single agent?**
- A When a task naturally splits into distinct specialized roles, benefits from parallelizing independent sub-tasks, or would overload a single agent's context.

- Q72 What is the orchestrator-worker pattern?**
- A A lead orchestrator agent decomposes the goal and delegates sub-tasks to specialized worker agents, then synthesizes their results.

- Q73 What is a 'blackboard' communication pattern?**
- A A shared memory space that multiple agents read from and write to, coordinating indirectly through shared state.

- Q74 What is a common failure mode at agent hand-offs?**
- A Context loss — a receiving agent lacking enough of the original context, leading to repeated work or contradicted decisions.

- Q75 Why are multi-agent systems harder to debug?**
- A Failures can originate from any agent's individual reasoning or from miscommunication between agents, multiplying possible failure points.

- Q76 What's a cost trade-off of multi-agent systems?**
- A Running multiple LLM instances and message-passing between them increases token usage, latency, and infrastructure cost versus a single agent.

- Q77 How can you reduce coordination failures between agents?**
- A Passing full, structured context at hand-offs, using a shared memory store, and clearly scoping each agent's responsibility.

Multi-Agent Systems

Questions 78–80 of 100

- Q78 What is role specialization in multi-agent design?**
- A** Assigning each agent a narrow, well-defined responsibility (e.g. researcher, writer, reviewer) instead of one agent doing everything.
- Q79 Should every complex task use multiple agents?**
- A** No — many tasks that seem to need multiple agents can be solved by a single agent with better tools and clearer instructions.
- Q80 What's an example of a good multi-agent use case?**
- A** A research-then-write pipeline where one agent gathers and verifies information while another focuses purely on drafting clear prose.

INTERVIEW PREP

Evaluation, Safety & Observability

Questions 81–86 of 100

Q81 Why is evaluating agents harder than evaluating single LLM outputs?

- A An agent's result depends on a full trajectory of decisions and tool calls, so failures can originate from many points, not just the final answer.

Q82 What is agent tracing?

- A Capturing a detailed step-by-step log of an agent's reasoning, tool calls, and results so runs can be replayed and debugged.

Q83 Name two common agent evaluation metrics.

- A Task success rate and tool-call accuracy (whether the right tool was called with correct arguments) are two common metrics.

Q84 What is LLM-as-judge evaluation?

- A Using a language model to grade another model's or agent's output against a rubric, at a scale manual grading can't match.

Q85 What is a guardrail?

- A A check — in code, a model, or explicit rules — that constrains what an agent is allowed to do or output.

Q86 Why should agent loops have step/cost limits?

- A Because the agent decides its own next steps, a bug or confusing scenario can cause excessive looping, wasting cost or taking unintended actions.

Evaluation, Safety & Observability

Questions 87–90 of 100

- Q87 What is sandboxing in agent execution?**
- A Running an agent's actions in an isolated environment with restricted permissions so mistakes or manipulation cause limited damage.
- Q88 What is the difference between input and output guardrails?**
- A Input guardrails screen what enters the model's context; output guardrails screen what the model produces before it is acted on or shown.
- Q89 What is prompt injection?**
- A Malicious instructions hidden in content the model reads (a webpage, file, tool output) attempting to hijack the model's behavior.
- Q90 What is the principle of least privilege in agent tooling?**
- A Granting an agent or tool only the minimum permissions necessary for its task, to limit potential damage from errors or manipulation.

INTERVIEW PREP

System Design & Scenario Questions

Questions 91–95 of 100

Q91 Design an agent to triage customer support tickets. What tools would it need?

- A Tools to read ticket details, search a knowledge base, look up customer/order history, and either draft a reply or escalate to a human — with human approval before sending or refunding.

Q92 How would you prevent a coding agent from running destructive shell commands?

- A Sandbox execution, maintain an allow-list of safe commands, require explicit human confirmation for anything destructive, and log every command run.

Q93 How would you design memory for an agent used daily by the same user over months?

- A A long-term store (vector or structured DB) for stable facts/preferences, session summaries written at the end of each conversation, and retrieval of only the most relevant memories per turn.

Q94 An agent keeps calling the wrong tool among two similar ones. How do you fix it?

- A Rewrite the tool descriptions to be more distinct, consider merging or renaming overlapping tools, and add few-shot examples showing correct selection.

Q95 How would you expose a company's internal database to AI assistants safely?

- A Build a scoped MCP server exposing only necessary, read-mostly operations, enforce authentication/authorization per user, and avoid exposing raw destructive queries directly.

System Design & Scenario Questions

Questions 96–100 of 100

Q96 How do you decide between a single agent and a multi-agent system for a research task?

- A** Start with a single agent with strong search/retrieval tools; move to multiple agents only if context gets overloaded or distinct specialized skills (e.g. verification vs. writing) genuinely improve quality.

Q97 How would you add observability to a production agent system?

- A** Log full reasoning traces and tool calls per run, track success/failure and cost metrics per task type, and set up alerting on abnormal loop lengths or error rates.

Q98 How would you handle a tool that occasionally returns huge amounts of data?

- A** Add pagination or summarization at the tool layer, cap the amount of raw data returned, and let the agent request more detail only if needed.

Q99 What would you check before granting an agent access to send real emails?

- A** That there's a human approval step for sending, rate limits are in place, the agent operates on a scoped account, and all outgoing content is logged.

Q100 How do you evaluate whether an MCP server is production-ready?

- A** Verify tool descriptions are clear and non-overlapping, inputs are validated, errors are handled gracefully, authentication is enforced, and the server has been tested against adversarial/unexpected inputs.

Glossary of Key Terms

Quick-reference definitions used throughout this book

Agent	An LLM-based system that perceives, reasons, and acts using tools in a loop to achieve a goal.
Agent Loop	The perceive → reason → act → observe cycle an agent repeats until a task is complete.
Chain-of-Thought (CoT)	A prompting technique where the model reasons step-by-step before answering.
Context Window	The maximum amount of text (tokens) a model can process at once.
Embedding	A numerical vector representation of text capturing its semantic meaning.
Function Calling	A model capability to request a structured call to a specific function with arguments.
Guardrail	A check that constrains what an agent is allowed to do or output.
Host (MCP)	The AI application a user interacts with, which runs MCP clients.
JSON-RPC	A lightweight remote procedure call protocol using JSON, used as MCP's message format.
MCP Client	A component that maintains a 1:1 connection to a specific MCP server.
MCP Server	A program that exposes tools, resources, and prompts to MCP clients.
Memory (Agent)	Information an agent retains, either within its context (short-term) or in external storage (long-term).
Multi-Agent System	An architecture where several specialized agents collaborate on a shared task.

Glossary of Key Terms

— continued —

Orchestrator	A lead agent or component that decomposes tasks and delegates to sub-agents or tools.
Prompt Injection	Malicious instructions hidden in content a model reads, aiming to hijack its behavior.
RAG (Retrieval-Augmented Generation)	Retrieving relevant external data and inserting it into context before generation.
ReAct	A pattern interleaving reasoning ('Thought') with tool actions and observations.
Reflexion	A self-critique pattern where an agent evaluates and improves its own output within a task.
Sampling (MCP)	An MCP mechanism letting a server request a completion from the client's LLM.
Sandboxing	Running agent actions in an isolated environment to contain potential damage.
Stopping Condition	The rule that tells an agent when to stop its loop (goal met, limit reached, etc.).
Tool Schema	A structured definition of a tool's name, description, and parameters.
Tree-of-Thought (ToT)	A reasoning pattern exploring multiple branches of thought in parallel.
Vector Store	A database storing embeddings for semantic similarity search.

Further Resources & Roadmap

What to do next

- ✓ Practice: build a single-tool agent end-to-end before attempting any multi-agent or MCP-heavy system.
- ✓ Read the official MCP specification and try building one small MCP server for a tool you personally use.
- ✓ Study open-source agent codebases (LangGraph examples, Claude Agent SDK samples) to see real tool/error handling.
- ✓ Set up basic tracing/logging on any agent you build — it is the highest-leverage debugging investment you can make.
- ✓ Learn evaluation basics: build even a 10-example test set with clear pass/fail criteria before shipping anything.
- ✓ Explore RAG fundamentals — chunking, embeddings, and re-ranking — since most real agents need grounded retrieval.
- ✓ Join communities and read changelogs for MCP and major agent frameworks; this space moves quickly.
- ✓ Revisit the 100 interview questions in this book periodically and try answering them without looking, out loud.

💡 **Tip:** The best way to learn agents and MCP is to build something real. Pick one use case and ship it this week.

THANK YOU FOR READING

Complete AI Agents & MCP Handbook

Created by

Kartik Mungole

@kartikmungole